Segundo parcial de Programación Web

Informe técnico — Aplicación web para un centro veterinario

Estudiante	Luis Carlos Amaya id 907274/ Sebastián Ruiz id 998667
Programa	Ingeniería de Sistemas
Asignatura	Programación Web
Docente	Luis Giovanni Mora Jaimes
Institución	UNIMINUTO
Fecha	17 de octubre de 2026

Contenido

1. Resumen y entregables 3

Entregables del parcial 3

Tecnologías 3

Cómo ejecutar el proyecto 3

2. Modelo de datos utilizado 4

2.1 Relaciones principales 4

2.2 Tablas 5

2.3 Integridad de los datos: «no debe existir información huérfana» 5

3. Endpoints implementados 6

3.1 Consulta de apoyo (combos y catálogos) 6

3.2 Citas médicas 6

3.3 Historias clínicas y anotaciones 7

3.4 Fórmulas médicas 7

3.5 Códigos de respuesta y formato de errores 7

4. Principales reglas de negocio 8

4.1 Citas médicas 8

4.2 Fórmulas médicas 9

4.3 Restricciones adicionales (decisiones propias) 9

4.4 Justificación de las restricciones de edición de citas 9

5. Decisiones técnicas relevantes 10

5.1 Arquitectura por capas en el backend 10

5.2 Manejo de errores centralizado 10

5.3 Filtros dinámicos con Specification (JPA Criteria) 10

5.4 Integridad en dos niveles 10

5.5 Vista resumida y detalle de fórmulas 10

5.6 Frontend Angular 10

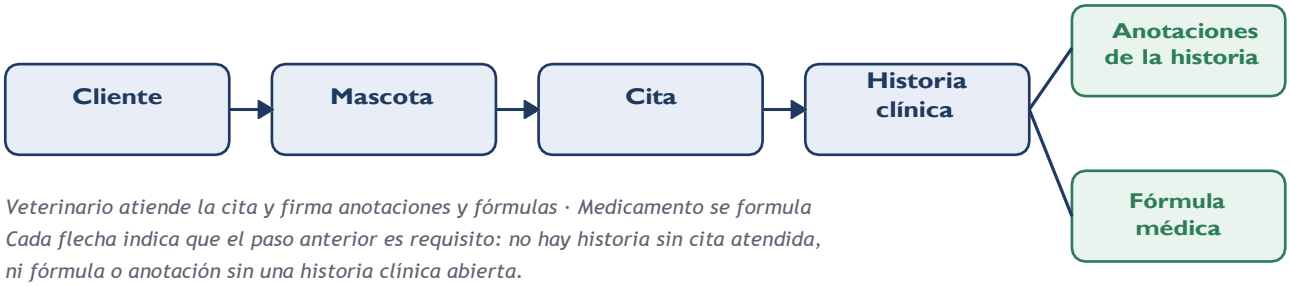
5.7 Combos dependientes 10

5.8 Fechas como texto ISO local 10

5.9 Configuración sin credenciales en el código 10

1. Resumen y entregables

La aplicación permite gestionar la atención de un centro veterinario siguiendo el flujo real de trabajo: un **cliente** tiene **mascotas**; la mascota solicita una **cita médica**; cuando la cita es atendida se registra una **historia clínica**, a la que el veterinario agrega **anotaciones** de seguimiento y **fórmulas médicas**. El backend expone una API REST y valida todas las reglas críticas; el frontend en Angular consume esa API y ofrece respuesta inmediata al usuario.



Veterinario atiende la cita y firma anotaciones y fórmulas · Medicamento se formula
Cada flecha indica que el paso anterior es requisito: no hay historia sin cita atendida, ni fórmula o anotación sin una historia clínica abierta.

Figura 1. Cadena de negocio de la aplicación.

Entregables del parcial

#	Entregable (punto 9 del enunciado)	Ubicación en el proyecto
1	Código fuente del backend en GitHub	`backend/`
2	Código fuente del frontend Angular en GitHub	`frontend-veterinaria/src/`
3	Script de la base de datos (SQL Server)	`database/01_clinica_veterinaria_sqlserver.sql`
4	Documento breve (este informe): modelo de datos, endpoints, reglas de negocio y decisiones técnicas	Secciones 2 a 5 de este documento

Tecnologías

Capa	Tecnología
Base de datos	Microsoft SQL Server (base **clinica_veterinaria**)
Backend	Java 17, Spring Boot 3.3.5, Spring Data JPA (Hibernate), Bean Validation
Frontend	Angular con componentes standalone, formularios reactivos, signals y carga diferida de rutas

Cómo ejecutar el proyecto

Paso	Acción
1	Ejecutar `database/01_clinica_veterinaria_sqlserver.sql` en SQL Server (crea la base, las tablas y datos de prueba).
2	Backend: definir `DB_PASSWORD` y ejecutar `mvn spring-boot:run` en `backend/` (puerto 8080).
3	Frontend: instalar dependencias con `npm install` y ejecutar `ng serve` (puerto 4200).

2. Modelo de datos utilizado

Se utilizó una base de datos **SQL Server** llamada **clinica_veterinaria**, adaptada del volcado MySQL *clinica* entregado con el parcial. Se rediseñaron las tablas de historias clínicas, anotaciones y fórmulas para ajustarlas al enunciado, y se agregaron restricciones que garantizan la integridad de los datos.

2.1 Relaciones principales

Relación	Cardinalidad	Significado
Cliente → Mascota	1 : N	Un cliente puede tener varias mascotas.
Raza → Mascota	1 : N	Cada mascota tiene una raza.
Especialización → Veterinario	1 : N	Cada veterinario tiene una especialidad.
Cliente / Mascota / Veterinario → Cita	1 : N	Una cita relaciona a una mascota de un cliente con un veterinario.
Cita → Historia clínica	1 : 1	Una cita atendida genera, como máximo, una historia clínica.
Historia clínica → Anotación	1 : N	Una historia acumula anotaciones de seguimiento, firmadas por un veterinario.
Historia clínica → Fórmula médica	1 : N	Una consulta puede originar varias fórmulas.
Medicamento → Fórmula médica	1 : N	Cada fórmula prescribe un medicamento del catálogo.

2.2 Tablas

Tabla	Descripción	Cambios respecto al dump original
`cliente`	Dueños de las mascotas.	Se eliminó *usuario_id* (no hace parte del parcial).
`raza`	Raza y especie.	Nombre único.
`mascota`	Pertenece a un cliente y a una raza.	Se agregó *activo*; llaves foráneas a cliente y raza.
`especializacion`	Especialidad del veterinario.	«Cardiología 2» se corrigió a «Cardiología».
`veterinario`	Profesionales que atienden (antes *medico*).	Renombrada porque el enunciado habla de veterinarios; se agregó *activo*.
`medicamento`	Catálogo de medicamentos.	Se agregó la restricción fecha de vencimiento ≥ fecha de compra; se actualizaron los vencimientos.
`cita`	Cita de una mascota con un veterinario.	Estados como texto con restricción CHECK; llaves foráneas a cliente, veterinario y mascota.
`historia_clinica`	Consulta realizada: cliente, mascota, cita, veterinario, fecha, motivo, diagnóstico, observaciones y estado (abierta/cerrada).	Antes solo tenía paciente y fecha; se rediseñó según el enunciado.
`anotacion_historia`	Seguimiento de una historia clínica.	Se agregaron *observación* y *diagnóstico de evolución*.
`formula_medica`	Fórmula emitida por un veterinario para una mascota.	Dosis, frecuencia y duración pasaron de texto libre a valores numéricos; se agregó *activo* para inactivar sin eliminar.

No se copiaron las tablas usuario y session del volcado original: no hacen parte del parcial y contenían credenciales y un token de sesión.

2.3 Integridad de los datos: «no debe existir información huérfana»

La integridad se garantiza en la propia base de datos, de modo que ni siquiera una escritura directa pueda dejar registros inconsistentes:

- **Llaves foráneas** en todas las relaciones, sin borrado en cascada.
- **Llaves foráneas compuestas** que amarran los datos entre sí: la mascota de una cita pertenece al cliente de la cita (`cita(mascota_id, cliente_id)`); la cita de una historia es de esa mascota (`historia(cita_id, mascota_id)`); y la historia de una fórmula es de la mascota de la fórmula (`formula(historia_id, mascota_id)`).
- **UNIQUE** sobre `cita_id` en la historia clínica: una cita genera una sola historia.
- **CHECK** en los estados de cita e historia, la dosis (> 0), la frecuencia (1–168 h), la duración (1–365 días) y las fechas del medicamento.

3. Endpoints implementados

URL base: `http://localhost:8080/api`. Todos los listados devuelven la estructura `{contenido, pagina, tamano, totalElementos, totalPaginas}` y aceptan los parámetros `pagina`, `tamano`, `orden` y `direccion` (asc/desc). El ordenamiento solo es posible por una lista blanca de columnas, lo que evita ordenar por campos arbitrarios.

3.1 Consulta de apoyo (combos y catálogos)

Método	Ruta	Descripción
GET	`/clientes`	Listar clientes (filtro *texto*, paginación).
GET	`/clientes/{id}`	Consultar un cliente.
GET	`/clientes/{id}/mascotas`	Mascotas de un cliente (alimenta el combo dependiente).
GET	`/mascotas/{id}`	Consultar una mascota.
GET	`/veterinarios`	Listar veterinarios activos.
GET	`/medicamentos`	Listar medicamentos vigentes (no vencidos).

3.2 Citas médicas

Método	Ruta	Descripción
GET	`/citas`	Listar citas. Filtros: cliente, mascota, clienteId, mascotaId, veterinarioId, estado, desde, hasta, sinHistoria. Orden: fechaHora (por defecto, descendente), cliente, mascota, veterinario, estado.
GET	`/citas/{id}`	Consultar una cita.
POST	`/citas`	Crear una cita.
PUT	`/citas/{id}`	Editar una cita (según su estado).
PATCH	`/citas/{id}/reactivar`	Reactivar una cita cancelada (operación explícita).

3.3 Historias clínicas y anotaciones

Método	Ruta	Descripción
GET	`/historias`	Listar historias. Filtros: cliente, mascota, motivo, clienteId, mascotaId, veterinarioId, estado, desde, hasta. Orden: fechaHora (por defecto, descendente), mascota, cliente, veterinario, motivo, estado.
GET	`/historias/{id}`	Consultar una historia.
POST	`/historias`	Crear una historia a partir de una cita atendida.
PATCH	`/historias/{id}/cerrar`	Cerrar la historia (queda de solo lectura).
GET	`/historias/{id}/anotaciones`	Anotaciones de la historia, de la más reciente a la más antigua. Filtros: texto, veterinarioId, desde, hasta. Orden: fechaHora, veterinario.
GET	`/historias/{id}/anotaciones/{anotacionId}`	Detalle de una anotación.
POST	`/historias/{id}/anotaciones`	Agregar una anotación (la fecha y hora las asigna el servidor).

3.4 Fórmulas médicas

Método	Ruta	Descripción
GET	`/formulas`	Listar fórmulas en **vista resumida**. Filtros: mascota, cliente, medicamento, mascotaId, veterinarioId, activo, desde, hasta. Orden: fechaCreacion (por defecto, descendente), fechaEmision, mascota, cliente, veterinario, medicamento, activo.
GET	`/formulas/{id}`	Detalle **completo** de una fórmula.
POST	`/formulas`	Crear una fórmula.
PUT	`/formulas/{id}`	Editar solo los datos permitidos (medicamento, dosis, frecuencia, duración e indicaciones).
PATCH	`/formulas/{id}/inactivar`	Inactivar una fórmula (no existe DELETE).
PATCH	`/formulas/{id}/activar`	Volver a activar una fórmula inactiva.

3.5 Códigos de respuesta y formato de errores

Código	Cuándo se usa
200 / 201	Consulta o edición exitosa / creación exitosa.
400	Campos inválidos u obligatorios vacíos; la respuesta incluye el detalle por campo.
404	El registro solicitado no existe (cliente, mascota, veterinario, cita, etc.).
405	Método no permitido (por ejemplo, intentar eliminar una fórmula).
409	Conflicto con el estado de los datos: cruce de horario, cita cancelada, historia cerrada, duplicados.
422	Datos válidos en formato pero inválidos para el negocio: mascota de otro cliente, fecha pasada, medicamento vencido.
500	Error inesperado del servidor.

Todos los errores usan el mismo cuerpo `{fecha, estado, error, mensaje, ruta, camposInvalidos}`. El mensaje es comprensible para el usuario y **nunca** expone trazas ni consultas SQL; el detalle técnico queda únicamente en el registro del servidor.

4. Principales reglas de negocio

Las reglas críticas se validan en el **backend**, en la capa de servicios, y se repiten en el frontend solo para dar respuesta inmediata. Además, las más importantes tienen una restricción equivalente en la base de datos como red de seguridad.

4.1 Citas médicas

#	Regla	Dónde se valida / resultado
1	El cliente debe existir.	ValidadorReferencias.clienteActivo → 404
2	La mascota debe pertenecer al cliente de la cita.	ValidadorReferencias.mascotaDelCliente → 422, y llave foránea compuesta en la BD
3	El veterinario debe existir.	ValidadorReferencias.veterinarioActivo → 404
4	Un veterinario no puede tener dos citas con menos de 15 minutos de diferencia.	CitaServiceImpl.validarDisponibilidad (ventana de ±15 min; ignora citas canceladas) → 409
5	Una mascota no puede tener dos citas simultáneas.	Misma ventana de ±15 min por mascota → 409 «Cita duplicada»
6	No se registran citas en fechas pasadas.	CitaServiceImpl.validarFechaFutura → 422
7	Estados válidos: Pendiente, Confirmada, Atendida, Cancelada y Programada.	Enumeración EstadoCita y restricción CHECK en la BD
8	Una cita cancelada no se modifica como si siguiera activa.	actualizar → 409; solo PATCH /reactivar puede reactivarla
9	Una cita atendida da origen a una historia clínica.	HistoriaServiceImpl.crear exige cita Atendida y sin historia previa
10	El backend valida las reglas críticas.	Todas las anteriores están en la capa de servicio; Angular solo da respuesta inmediata

4.2 Fórmulas médicas

#	Regla	Dónde se valida / resultado
1–2	La mascota existe y pertenece a un cliente registrado y activo.	FormulaServiceImpl.crear → 404 / 422
3	El veterinario existe.	ValidadorReferencias.veterinarioActivo
4	La emisión no puede ser anterior a la consulta ni futura.	crear → 422; el formulario además bloquea fechas fuera de rango
5	Dosis, frecuencia y duración válidas.	Bean Validation en FormulaRequest (dosis 0,01–10 000; frecuencia 1–168 h; duración 1–365 días), CHECK en la BD y validadores en Angular
6	Campos obligatorios en frontend y backend.	Validators.required en Angular; @NotNull / @NotBlank → 400 con camposInvalidos
7	No asociar la fórmula a una consulta de otra mascota.	crear → 422, llave foránea compuesta en la BD, y el combo solo lista las consultas de la mascota
8	Sin eliminación física.	No existe DELETE; se usa PATCH /inactivar y /activar

4.3 Restricciones adicionales (decisiones propias)

- Una **historia cerrada** es de solo lectura: no admite anotaciones ni fórmulas nuevas (409).
- No puede haber **dos fórmulas activas del mismo medicamento** en una misma consulta, para evitar duplicidad.
- Un **medicamento vencido** no se puede formular.
- La **fecha y hora de una anotación** las asigna el servidor, de modo que el historial no pueda alterarse.
- La fecha y hora de una **historia** no pueden ser futuras.

4.4 Justificación de las restricciones de edición de citas

Estado	Qué se permite y por qué
Pendiente, Confirmada, Programada	Se puede cambiar fecha, hora, veterinario y motivo; cada cambio vuelve a validar cruces y fecha. El estado solo avanza por transiciones válidas.
Cancelada	No se modifica como si siguiera activa, porque otras citas pudieron ocupar ese horario. Solo se vuelve a activar con la operación explícita **Reactivar**, que vuelve a comprobar fecha futura y disponibilidad.
Atendida	Es información histórica y de ella depende la historia clínica; modificarla alteraría el registro médico. Es un estado terminal.
Transición a Atendida	Solo desde Confirmada o Programada (no desde Pendiente): una cita atendida debió haber sido confirmada antes.

5. Decisiones técnicas relevantes

5.1 Arquitectura por capas en el backend

`controller` (HTTP) → `service` (interfaz) → `service/impl` (reglas de negocio) → `repository` (Spring Data JPA) → `entity` (tablas). Los *dto* separan lo que viaja por la red de las entidades, y el paquete *exception* centraliza los errores. Cada capa tiene una sola responsabilidad.

5.2 Manejo de errores centralizado

Un `GlobalExceptionHandler` (`@RestControllerAdvice`) traduce cada excepción a un código HTTP y a un mensaje uniforme, sin exponer detalles internos.

5.3 Filtros dinámicos con Specification (JPA Criteria)

Un solo método de consulta combina cualquier conjunto de filtros, con paginación y ordenamiento **en la base de datos**, no en memoria. El orden usa una lista blanca de columnas y desempata por *id* para que las páginas sean estables.

5.4 Integridad en dos niveles

Las reglas viven en los servicios (con mensajes claros) y, además, en la base de datos (llaves foráneas compuestas, CHECK y UNIQUE) como red de seguridad.

5.5 Vista resumida y detalle de fórmulas

El listado devuelve *FormulaResumenResponse* (sin indicaciones, historia ni datos de auditoría); el detalle devuelve *FormulaDetalleResponse* completo. Así el listado es liviano y la información completa se pide solo cuando se necesita.

5.6 Frontend Angular

Componentes standalone con carga diferida de rutas; formularios reactivos con validación; servicios dedicados al consumo del API; un componente de tabla reutilizable (*TablaDatos*) con su control de paginación y orden (*ControlTabla*) usado en las cuatro tablas de la aplicación; utilidades para fechas, errores y validadores; y estado de pantalla con *signals* (indicadores de carga y mensajes de éxito y error).

5.7 Combos dependientes

Cliente → Mascotas del cliente → Cita atendida sin historia (en historias) o Consulta abierta (en fórmulas). Cada combo consulta al backend únicamente lo que corresponde al valor anterior.

5.8 Fechas como texto ISO local

El frontend maneja las fechas como `yyyy-MM-ddTHH:mm` para evitar corrimientos por zona horaria al mostrarlas.

5.9 Configuración sin credenciales en el código

La conexión a SQL Server se define con variables de entorno (`DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`); la clave no se guarda en el repositorio. CORS está limitado al origen `http://localhost:4200`.